Development of a Cross-Dataset Generalisable Intrusion Detection Framework Using Distributed Big-Data Analytics

DTS 301: Big Data Computing | Redeemer's University Nigeria | Group members: Bankole Oluwanifemi Ogo-oluwakitan (RUN/CMP/24/17179); Bunmi Edith Adeola (RUN/CMP/24/17180); Dada Temiloluwa (RUN/CMP/24/17183); Daramola Eyinjuoluwa Hephzibah (RUN/CMP/24/17184); David Esther Ebunrereoluwa (RUN/CMP/24/17185); Diya Oluwatobi Oyefeso (RUN/CMP/24/17186); Ebenuwah David Obichukwu (RUN/CMP/24/17187) | November 2026

Abstract

Machine-learning intrusion detectors are usually evaluated on the data they were trained on, which hides how they behave on a different network. We built a distributed pipeline on HDFS, Spark MLlib and MongoDB that cleans, harmonises and models two public datasets, CICIoT2023 [1] and UNSW-NB15 [2], and we measured the generalisation gap between them. After removing duplicates and invalid rows, only five features could be matched across the datasets. With these features, a Random Forest reaches a balanced accuracy of 0.98 on CICIoT2023 and 0.79 on UNSW-NB15 when trained and tested on the same dataset, but only 0.44 to 0.50 (chance level) when trained on one dataset and tested on the other. Class-conditional medians show why: packet rate and TTL point in opposite directions in the two datasets. Rescaling, rank normalisation and feature removal did not close the gap, and a multiclass task failed in the same way. About 100 labelled rows from the new dataset, however, recover 96% to 98% of in-dataset performance. The pipeline's runtime grew roughly linearly with data size on a single laptop, up to 6.1 million clean rows. We recommend a short labelled onboarding step for each new environment.

1. Introduction

1.1 Motivation

Network intrusion detection systems use machine learning to separate attack traffic from normal traffic. Published results are often close to perfect, but they usually come from training and testing on the same dataset [4], [10]. A deployed detector meets different devices, protocols and attack tools, so the question that matters is whether a model trained in one environment still works in another. Network traffic is also big data: CICIoT2023 alone contains tens of millions of records [1], which calls for distributed storage and processing [13], [15].

1.2 Objectives

1. Build a distributed pipeline for two public intrusion datasets.
2. Harmonise the datasets into one shared feature and label scheme.
3. Measure how well models generalise from one dataset to the other (the generalisation gap).
4. Test whether the gap can be closed.
5. Show that the pipeline scales with data size.

1.3 Contributions

- A reproducible pipeline on HDFS, Spark MLlib and MongoDB, with all scripts and documentation in the project repository.
- A documented harmonisation of two datasets that describe different units of observation (packet windows and network flows), with the reasons for each choice.
- A quantified generalisation gap with diagnostics: balanced accuracy, AUC, class-conditional medians, feature ablation and three random seeds.
- An adaptation experiment showing how many labelled target rows are needed to close the gap.
- Scalability measurements for the full pipeline.

2. Related Work

2.1 Intrusion detection datasets and their problems

Early research relied on KDD Cup 99, whose analysis found a large number of redundant records that bias learning [17]. Later datasets such as CICIDS2017 [6] and UNSW-NB15 [2], [3] were built to be more realistic, and surveys catalogue their properties [5]. Even recent datasets contain errors: Engelen et al. found problems with traffic generation, flow construction, feature extraction and labelling in CICIDS2017, and more than 20% of its traffic traces were reconstructed or relabelled [7]. CICIoT2023 [1] extends the field to IoT networks, with 105 devices and 33 attacks in seven categories, all launched by malicious IoT devices. Our finding that duplicates make up roughly a third of both datasets is consistent with this concern about data quality.

Sommer and Paxson argue that intrusion detection is a harder setting for machine learning than most applications and that evaluation needs particular care [4]. Arp et al. list common pitfalls in machine learning for security, including spurious correlations, inappropriate performance measures and the base rate fallacy [10]. We meet all three in this study (Section 7).

2.2 Cross-dataset evaluation and common feature sets

A recurring obstacle is that each dataset ships its own feature set. Sarhan et al. argue that this makes comparison across datasets, and therefore assessment of generalisation, very difficult, and they propose NetFlow-based standard feature sets of 12 and 43 features [8]. In a follow-up they compare a NetFlow feature set with a CICFlowMeter feature set across three datasets (CSE-CIC-IDS2018, BoT-IoT and ToN-IoT) and find that the NetFlow set gives better detection accuracy [9].

2.3 Transfer and domain adaptation

Ben-David et al. give a theory that relates performance on a target domain to the error on the source domain and the difference between the two domains [11]. Pan and Yang survey transfer learning, including settings where a small amount of labelled target data is available [12]. Our adaptation experiment (Section 6.5) is a simple instance of the latter.

2.4 Distributed big-data frameworks

HDFS provides replicated distributed storage [15]. Apache Spark offers in-memory distributed processing [13], and MLlib supplies scalable learning algorithms [14], including Random Forests [16].

2.5 Position of this work

We worked from the feature files published with each dataset instead of re-extracting features from packet captures. This is why only five features could be shared, and it makes the study a test of what can be done with published features. Re-extracting both datasets with a common tool, as in [8], is the more robust route and is listed as future work.

3. Datasets

3.1 Overview

	CICIoT2023 [1]	UNSW-NB15 [2]
Producer	Canadian Institute for Cybersecurity, University of New Brunswick	Cyber Range Lab, UNSW Canberra
Traffic	105 IoT devices, 33 attacks in 7 categories	Mixed normal and attack traffic, 9 attack types
Raw size	63 merged CSV files, about 46.7 million records reported for the dataset	2,540,044 records in 4 CSV files; published split of 175,341 training and 82,332 testing records
One row describes	A window of packets (39 features plus a label in the merged CSV)	One network flow (45 columns in the split files)
Used here	5 files for modelling (3,228,175 rows); up to 15 files for scaling tests	Published training and testing sets

Files used for CICIoT2023 modelling: Merged01, Merged15, Merged30, Merged45 and Merged63, chosen from across the range so the sample is not limited to one part of the capture. The sample contains all 34 labels, although rare classes are small (for example 85 rows of the uploading attack).

3.2 Data quality

The cleaning step removed exact duplicate rows and rows with missing, NaN or infinite values. Duplicates were a large share of both datasets, mostly repeated flood traffic.

Dataset	Raw rows	After duplicates removed	Final	Removed
CICIoT2023 (5 files)	3,228,175	2,017,860	2,017,802	37.5%
UNSW-NB15 training	175,341	107,740	107,740	38.6%
UNSW-NB15 testing	82,332	55,945	55,005	33.2%

Two further problems were found and fixed. First, 940 UNSW-NB15 testing rows were identical to training rows and were removed from the testing set, because they would inflate test scores. Second, the CICIoT2023 `rate` column writes infinity as the text inf, so Spark read the column as text and the first cleaning filter skipped it; 16 rows with infinite values were removed after the column was converted to numbers. A first download of the UNSW-NB15 training file was also truncated (67,688 of 175,341 rows) and was re-downloaded and checked.

3.3 Class balance

Dataset	Benign	Attack	Benign share
CICIoT2023 (cleaned sample)	75,655	1,942,147	3.7%
UNSW-NB15 training	51,890	55,850	48.2%
UNSW-NB15 testing	34,206 (before overlap removal)	21,739	61.1%

The base rates differ greatly, so plain accuracy is not a meaningful measure here: a model that always answers attack scores about 96% on CICIoT2023.

4. System Design and Implementation

4.1 Architecture

All services run in Docker containers on one machine: HDFS (one namenode, one datanode, Hadoop 3.3.6), Spark 3.5.1 in standalone mode (one master, one worker with 4 cores and a 4 GB executor), and MongoDB 7. The data flows as follows:

1. Raw CSV files are uploaded to HDFS.
2. Spark cleans them and writes Parquet files.
3. A harmonisation job builds the shared features and unified labels and writes a second set of Parquet files.
4. Spark MLlib jobs run the experiments and write results as JSON to HDFS.
5. A loader script copies the results into MongoDB (collections for experiment results and for scalability runs).

The Spark image was extended with numpy, which MLlib requires. Parquet was used for intermediate data because it is compressed, columnar and keeps column types.

4.2 Harmonisation

CICIoT2023 features describe a window of packets, while UNSW-NB15 features describe a whole flow, with source and destination values kept separate. Only features that can be expressed on both sides were kept. We checked each candidate against the published feature descriptions and against the data.

Shared feature	CICIoT2023 source	UNSW-NB15 source
rate	rate (packets per second in the window)	rate, equal to (spkts + dpkts - 1) / dur, checked against sample rows
ttl	time_to_live	sttl
avg_pkt_size	avg (mean packet length)	(sbytes + dbytes) / (spkts + dpkts)
proto_tcp, proto_udp	protocol_type equal to 6 or 17	proto equal to tcp or udp

Findings from the checks that shaped the feature set:

- CICIoT2023 `number` (packets in the window) is capped at 100 and was dropped.
- ICMP is almost absent in UNSW-NB15 and GRE appears only in CICIoT2023, so ICMP and other-protocol indicators could act as shortcuts. They were excluded from the main feature set.
- SSH, SMTP and IRC indicators are present in under 0.5% of CICIoT2023 rows and were dropped.
- Inter-arrival time is in seconds in CICIoT2023 and milliseconds in UNSW-NB15, and the medians still differ by about 300 times, so it was not used.

- Source and destination addresses, ports and timestamps in the raw UNSW-NB15 files were excluded, because they identify hosts and times rather than behaviour.

Labels were mapped to a binary label (benign or attack) and to four unified classes:

Unified class	CICIoT2023 labels	UNSW-NB15 category
BENIGN	BENIGN	Normal
DOS_DDOS	all DDoS, DoS and Mirai flood labels	DoS
RECON	reconnaissance labels and vulnerability scan	Reconnaissance
OTHER_ATTACK	spoofing, brute force, web-based attacks, backdoor	all other categories

The final harmonised data holds 2,017,802 CICIoT2023 rows and 162,745 UNSW-NB15 rows. Five features cannot separate every row: 0.05% of CICIoT2023 rows and 4.7% of UNSW-NB15 rows have a feature vector that appears with both benign and attack labels, which limits accuracy on UNSW-NB15.

4.3 Models

We used Logistic Regression and Random Forest from Spark MLlib [14], [16]. Experiment A used 50 trees of depth 10 and 50 iterations for logistic regression. Experiments B to D used a forest of 30 trees of depth 8. Training sets were balanced by undersampling the majority class.

5. Experimental Methodology

- **Splits.** CICIoT2023 was split at random, 70% for training and 30% for testing. UNSW-NB15 used its published training and testing sets.
- **Cross-dataset tests.** A model trained on one dataset was tested on the same test set that was used for the within-dataset baseline of the other dataset, so the gap is measured on identical data.
- **Metrics.** Balanced accuracy (the mean of attack recall and benign recall; 0.5 is chance) is the main metric, with AUC as a second. Macro-F1 is reported for completeness but depends on the base rate of each test set.

- **Experiment A.** Binary detection, two models, raw features against log transform plus per-dataset standardisation.
- **Experiment B.** AUC, rank normalisation (each value replaced by its percentile within its own dataset) and leave-one-feature-out ablation.
- **Experiment C.** Three random seeds, a jointly trained model, and a multiclass task on the shared classes (benign, DoS/DDoS, reconnaissance).
- **Experiment D.** Few-shot adaptation: the source training set plus a small class-balanced sample of labelled target rows, weighted so that the target rows count as much as the source data.
- **Scalability.** One end-to-end job (read and clean, build features, train and evaluate a forest) run on 1, 5, 10 and 15 CICIoT2023 files and with 1, 2 and 4 cores. The machine has 4 logical cores and 7.7 GiB of memory.

6. Results

6.1 Experiment A: within-dataset and cross-dataset performance

Balanced accuracy, single run:

Preprocessing and model	Within CIC	Within UNSW	UNSW to CIC	CIC to UNSW
raw features, Logistic Regression	0.915	0.692	0.303	0.328
raw features, Random Forest	0.977	0.802	0.456	0.461
log and standardise, Logistic Regression	0.956	0.690	0.172	0.495
log and standardise, Random Forest	0.977	0.809	0.492	0.483

Within a dataset the detector works. Across datasets it sits at or below chance, and the models mostly answer attack: for the Random Forest with log and standardisation, attack recall is 0.87 to 0.97 while benign recall is 0.01 to 0.10. Macro-F1 for the Random Forest falls from 0.82 and 0.76 to 0.47 and 0.45 across datasets (raw features), but macro-F1 also depends on the base rate: within CICIoT2023 it is 0.82 despite a balanced accuracy of 0.98, because benign traffic is only 4% of the test set.

6.2 Why the models fail to transfer

Median feature values by class:

Dataset	Class	rate	ttl	avg_pkt_size	TCP share	UDP share
CICIoT2023	benign	171	98.9	247.9	0.916	0.082
CICIoT2023	attack	16,787	64	60.0	0.569	0.240
UNSW-NB15	benign	943	31	109.2	0.754	0.235
UNSW-NB15	attack	43.9	254	100.0	0.677	0.192

Attacks have a much higher rate than benign traffic in CICIoT2023 (about 100 times) and a much lower rate in UNSW-NB15 (about 20 times lower). TTL is lower for attacks in CICIoT2023 (64 against 99) and much higher in UNSW-NB15 (254 against 31). Packet size separates the classes strongly in CICIoT2023 but hardly at all in UNSW-NB15. A model trained on one dataset therefore learns rules that point the wrong way on the other. This matches the AUC values: for raw features trained on UNSW-NB15 and tested on CICIoT2023, the AUC over three seeds is 0.149 (standard deviation 0.071), so the ranking of traffic is systematically reversed.

6.3 Experiment B: preprocessing and feature ablation

Random Forest, cross-dataset tests, balanced accuracy / AUC (single run):

Preprocessing	Features	UNSW to CIC	CIC to UNSW
log and standardise	all five	0.505 / 0.772	0.476 / 0.618
log and standardise	without rate	0.477 / 0.569	0.567 / 0.549
log and standardise	without ttl	0.170 / 0.192	0.492 / 0.545
log and standardise	without avg_pkt_size	0.419 / 0.257	0.443 / 0.504
log and standardise	without protocol	0.498 / 0.308	0.496 / 0.546
rank	all five	0.192 / 0.248	0.461 / 0.341
rank	without rate	0.187 / 0.246	0.528 / 0.424
rank	without ttl	0.287 / 0.264	0.522 / 0.561
rank	without avg_pkt_size	0.194 / 0.170	0.455 / 0.392

Preprocessing	Features	UNSW to CIC	CIC to UNSW
rank	without protocol	0.201 / 0.190	0.340 / 0.282

No variant transferred reliably. The one AUC clearly above chance (0.772) was not stable: repeated over three seeds, the same configuration gave AUC values of 0.878, 0.098 and 0.563.

6.4 Experiment C: seeds, joint training and multiclass

Binary Random Forest, mean over three seeds (balanced accuracy, with standard deviation for the cross-dataset cases):

Preprocessing	Within CIC	Within UNSW	UNSW to CIC	CIC to UNSW
raw features	0.976	0.783	0.495 (0.042)	0.441 (0.008)
log and standardise	0.976	0.788	0.484 (0.036)	0.476 (0.002)

A model trained on both datasets together performs as well as the specialised models: balanced accuracy 0.974 on CICIoT2023 and 0.783 on UNSW-NB15, against 0.976 and 0.788. The datasets are therefore compatible when a model sees both, even though they do not transfer. This is not generalisation to an unseen dataset.

The multiclass task on the shared classes fails in the same way. Macro recall is 0.816 within CICIoT2023 and 0.841 within UNSW-NB15, but 0.387 (UNSW-NB15 to CICIoT2023) and 0.377 (CICIoT2023 to UNSW-NB15) across datasets, against a chance level of 0.333. The cross-dataset models collapse to one class: DoS/DDoS for UNSW-NB15 to CICIoT2023, mostly reconnaissance for the other direction.

6.5 Experiment D: few-shot adaptation

Balanced accuracy, mean over three seeds, against the number of labelled target rows added to training:

Labelled target rows	UNSW to CIC	CIC to UNSW
0	0.484	0.476
about 100	0.953	0.754

Labelled target rows	UNSW to CIC	CIC to UNSW
about 500	0.967	0.756
about 2,000	0.971	0.774
about 10,000	0.973	0.771
Within-dataset reference	0.976	0.788

About 100 labelled rows recover 98% (CICIoT2023 as target) and 96% (UNSW-NB15 as target) of in-dataset performance, and more labels add little. On UNSW-NB15 benign recall stays around 0.5 to 0.6 even with 10,000 labelled rows, consistent with the 4.7% of ambiguous rows.

6.6 Scalability

Runtime against data size, 4 cores:

Input files	Input size	Clean rows	Clean (s)	Features (s)	Train (s)	Total (s)
1	147 MB	544,929	68.0	4.1	32.5	104.5
5 (fastest of 6 runs)	739 MB	2,207,344	157.3	4.9	57.5	219.7
10	1,510 MB	4,128,223	244.5	7.2	68.8	320.5
15	2,364 MB	6,144,389	374.3	9.1	95.7	479.1

From 10 to 15 files the input grows 1.57 times and the runtime 1.49 times, so growth is close to linear, at roughly 12,800 clean rows per second. Cleaning (parsing and duplicate removal) takes 76% to 78% of the time at 10 and 15 files. Extrapolating to all 63 files (about 26 million clean rows) gives roughly 35 minutes; this is an estimate, not a measurement. Model quality was stable across sizes (balanced accuracy 0.973 to 0.980).

Runtime against cores, 5 files:

Cores	Total time (s)	Speedup against 1 core
1	283.6	1.00
2	207.7	1.37
4	six runs: 219.7 to 1,100.6 (median 394.5)	0.72 (median), 1.29 (fastest), 0.26 (slowest)

Going from one to two cores gave a 1.37 times speedup. Four cores gave no further gain: six runs took between 220 and 1,101 seconds (median 394.5 seconds), slower than the one-core and two-core runs, each of which is a single measurement. Stopping MongoDB for three of the four-core runs did not make them faster. The cause is unknown. Possible explanations, none of them tested, are that the four logical cores are shared with HDFS, Docker and Windows, and that four concurrent tasks sharing a 4 GB executor spill heavily to disk (the machine has 7.7 GiB of memory and swap was in use).

7. Discussion

7.1 Why generalisation fails

The failure is systematic, not random. The relationship between two of the three continuous features and the attack label is inverted between the datasets, and a third feature separates the classes in only one of them. The TTL difference probably reflects which devices generated the traffic in each testbed rather than anything about the attacks, which would make it a spurious correlation that helps within a dataset and hurts across datasets. A second contributor is the unit of observation: CICIoT2023 rows summarise windows of packets, while UNSW-NB15 rows describe whole flows, so the same feature name measures different things.

7.2 Links to known pitfalls

Three pitfalls described in [10] appear in this study. Spurious correlation: TTL and rate encode the testbed. Inappropriate performance measures: plain accuracy would report about 96% for a detector that always answers attack, and macro-F1 changes with the base rate of the test set, so we used balanced accuracy and AUC. Base rate: benign traffic is 4% of CICIoT2023 and about half of UNSW-NB15, so the two test sets are not comparable on threshold-dependent metrics without correction.

7.3 What works

Two things closed the gap. Training one model on both datasets removed it on the tests that model had seen, and adding about 100 labelled rows from the new dataset recovered 96% to 98% of in-dataset performance. The practical recommendation is an onboarding step for each new environment: label a few hundred rows and retrain, or pool data from several environments. Common feature sets such as those proposed in [8] and [9] address the cause more directly and are the natural next step.

7.4 Scalability

The pipeline's runtime grows close to linearly with data size on one laptop, which is the behaviour needed for the full 63-file dataset. Extra cores helped from one to two but not beyond, for reasons we did not establish. Showing horizontal scaling would need a multi-node cluster.

8. Limitations

- Only five features could be shared, and CICIoT2023 describes windows while UNSW-NB15 describes flows.
- The CICIoT2023 test split is a random split of windows from the same captures, so within-dataset scores are probably optimistic, and this probably flatters the adaptation results with CICIoT2023 as the target. The UNSW-NB15 results use the published split and are the safer estimate.
- Modelling used 5 of 63 CICIoT2023 files.
- Experiments A and B are single runs. Experiment C and D use three seeds, which vary only the split and the undersampling.
- The adaptation experiment uses a class-balanced labelled sample and a weighting scheme chosen by the authors. It is supervised adaptation, not generalisation to an unseen dataset.
- Scalability was measured on one machine with one worker, mostly with a single run per configuration, and run-to-run variation was large.

9. Conclusion and Future Work

We built a reproducible distributed pipeline on HDFS, Spark and MongoDB and used it to measure cross-dataset generalisation between CICIoT2023 and UNSW-NB15. Within a dataset, detection works (balanced accuracy 0.98 and 0.79). Across datasets, with the

five features that could be shared, it fails (0.44 to 0.50), because the relationship between the features and the attack label is inverted between the two environments. A small labelled sample from the new environment fixes most of the problem, and the pipeline scales close to linearly with data size.

Future work: (1) re-extract features from the packet captures of both datasets with a common tool to obtain a larger shared feature set [8]; (2) aggregate UNSW-NB15 flows into packet windows so that both datasets describe the same unit of observation; (3) split CICIoT2023 by capture file instead of at random; (4) repeat the experiments with more datasets and more seeds; (5) test on a multi-node cluster.

Group Contributions

[To be completed by the group so that it reflects what each member actually did.]

Member	Role	Contribution
Bankole Oluwanifemi Ogo-oluwakitan (RUN/CMP/24/17179)	[role]	[what they did]
Bunmi Edith Adeola (RUN/CMP/24/17180)	[role]	[what they did]
Dada Temiloluwa (RUN/CMP/24/17183)	[role]	[what they did]
Daramola Eyinjuoluwa Hephzibah (RUN/CMP/24/17184)	[role]	[what they did]
David Esther Ebunrereoluwa (RUN/CMP/24/17185)	[role]	[what they did]
Diya Oluwatobi Oyefeso (RUN/CMP/24/17186)	[role]	[what they did]
Ebenuwah David Obichukwu (RUN/CMP/24/17187)	[role]	[what they did]

References

[1] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu and A. A. Ghorbani, 'CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,' Sensors, vol. 23, no. 13, art. 5941, 2023. doi:10.3390/s23135941

[2] N. Moustafa and J. Slay, 'UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set),' in Military Communications and Information Systems Conference (MilCIS), 2015.

[3] N. Moustafa and J. Slay, 'The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set,' Information Security Journal: A Global Perspective, vol. 25, 2016.

[4] R. Sommer and V. Paxson, 'Outside the closed world: On using machine learning for network intrusion detection,' in IEEE Symposium on Security and Privacy, 2010.

[5] M. Ring, S. Wunderlich, D. Scheuring, D. Landes and A. Hotho, 'A survey of network-based intrusion detection data sets,' Computers & Security, vol. 86, pp. 147-167, 2019.

[6] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, 'Toward generating a new intrusion detection dataset and intrusion traffic characterization,' in Proc. 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018.

[7] G. Engelen, V. Rimmer and W. Joosen, 'Troubleshooting an intrusion detection dataset: the CICIDS2017 case study,' in IEEE Security and Privacy Workshops (SPW), 2021, pp. 7-12. doi:10.1109/SPW53761.2021.00009

[8] M. Sarhan, S. Layeghy and M. Portmann, 'Towards a standard feature set for network intrusion detection system datasets,' Mobile Networks and Applications, 2022. doi:10.1007/s11036-021-01843-0

[9] M. Sarhan, S. Layeghy and M. Portmann, 'Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection,' Big Data Research, 2022. doi:10.1016/j.bdr.2022.100359

[10] D. Arp et al., 'Dos and don'ts of machine learning in computer security,' in USENIX Security Symposium, 2022.

[11] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira and J. W. Vaughan, 'A theory of learning from different domains,' Machine Learning, vol. 79, pp. 151-175, 2010.

[12] S. J. Pan and Q. Yang, 'A survey on transfer learning,' IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345-1359, 2010.

[13] M. Zaharia et al., 'Apache Spark: a unified engine for big data processing,' Communications of the ACM, vol. 59, no. 11, pp. 56-65, 2016.

[14] X. Meng et al., 'MLlib: machine learning in Apache Spark,' Journal of Machine Learning Research, vol. 17, no. 34, pp. 1-7, 2016.

[15] K. Shvachko, H. Kuang, S. Radia and R. Chansler, 'The Hadoop distributed file system,' in IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), 2010.

[16] L. Breiman, 'Random forests,' Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[17] M. Tavallaee, E. Bagheri, W. Lu and A. A. Ghorbani, 'A detailed analysis of the KDD CUP 99 data set,' in IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), 2009.

Appendix A: Reproducing the Results

The repository README gives the full procedure: data to download, cluster set-up with Docker Compose, the order of the Spark jobs with approximate run times, how the results are loaded into MongoDB, and fixes for the problems met during the project. Supporting notes are in the docs folder: dataset sources and citations, the cleaning log, the harmonisation plan and the full results. Repository link: [add link].